

COMP3329 - Computer game design and programming

Game Report



Group 24

Chau Wai Yee (3036336168)

Chung Ka Yi (3036058247)

Hui Lok (3036059980)

Lam Wui Yan (3036002420)

Contents

1. Overview
2. Scope of Project
 - 2.1 Initial Game Idea / Inspiration
 - 2.2 Feature Scope
 - 2.3 Graphics and Character Designing
 - 2.4 Coding
3. Game Elements
 - 3.1 Character Designs (Abstract of sprites)
 - 3.2 Environments / Levels / Screens
 - 3.2.1 Opening, Ending, Start Menu, Buttons and Panels
 - 3.2.2 Round 1 & 3: Side-Scrolling Campus (Rescue)
 - 3.2.3 Round 2: Fixed Defense
 - 3.2.4 Round 4: Boss Arena
 - 3.2.5 Game Over / Ending Screen
 - 3.3 UI, Controls, and Feedback
 - 3.4 Audio
4. Evaluation: The Four Elements (Jesse Schell)
 - 4.1 Mechanics
 - 4.2 Aesthetics
 - 4.3 Story
 - 4.4 Technology
5. Gameplay Testing and Issue Analysis
 - 5.1 Testing Methodology
 - 5.2 Issue Analysis & Resolutions
6. Future Plan
 - 6.1 Technical Upgrades and Refinements
 - 6.2 Advanced Level Design & Environmental Mechanics
7. Conclusion

References / Credits

1. Overview

Rapocalypse – GPA Saver is a 2D action-platformer game inspired by the rubber-hose animation style of Cuphead and early 20th-century American cartoons. In this game, player controls Joe, a student battling an "apocalypse" of rabbit enemies that represent the overwhelming stress of university life. The gameplay alternates between side-scrolling rescue missions and fixed-position defense, culminating in a multi-phase boss fight. The GPA health bar starts at 4.3 and decreases with each hit, directly linking survival to academic performance.

3 Key Original Contributions:

- **Artistic Synthesis of Era and Locale:** Successfully fused the 1930s "rubber-hose" animation aesthetic with the localized architecture and culture of the University of Hong Kong (HKU) campus.
- **Multimodal Level Architecture:** Engineered a diverse gameplay loop that shifts from side-scrolling rescue missions (Rounds 1 and 3) to a fixed-position defense stage (Round 2), culminating in a multi-phase boss arena (Round 4).
- **High-Skill Movement and Rescue Systems:** Developed an advanced traversal system featuring "double dash-jumps" paired with proximity-based rescue triggers, creating a layered difficulty curve that requires players to master platforming precision alongside tactical enemy prioritization.

2. Scope of Project

2.1 Initial Game Idea / Inspiration

Our team sought to replicate the nostalgic yet intense atmosphere of Cuphead, focusing on a "survival-style" shooting mechanic where players must master enemy patterns. By grounding the setting in the University of Hong Kong campus, we transformed abstract academic pressures into a literal invading force of rabbits that Joe must defeat to protect his grades during midterm, assignments and examination.

2.2 Feature Scope

During the planning stage of our game design, a wide array of student-themed obstacles and rabbit behaviors were brainstormed. However, due to time constraints, we refined the scope to focus on mechanics that enhance the core 'academic stress' theme. Selected features include one playable character (Joe) and four distinct rabbit enemy types: Mover, Chaser, Flyer, and the Rabbit Queen boss. To maintain a high-quality experience, we simplified enemy patterns by removing the 'Burrower Rabbit' and prioritized the GPA

health bar as the critical thematic element over secondary systems like the Power Bar or coin collection.

During gameplay, players navigate four distinct rounds designed to test different skills. In the side-scrolling rescue rounds (1 and 3), players can utilize a Dash-Jump (Shift + Space) to explore higher platforms and save more students trapped in bubbles. Round 2 is a fixed-defense survival stage, while Round 4 features a multi-phase boss battle where the Rabbit Queen's behaviors are managed through Animator State Transitions and Trigger-based Logic. Control optimization was also implemented, changing the shooting mechanic to a mouse click for more comfortable and efficient control, and adding an ESC key pause menu to improve user accessibility.

A total of four rounds were implemented, transitioning from curated static levels to a dynamic, randomly spawned enemy wave finale. Level 4 utilizes an infinite scroller script to create a seamless background transition during the boss fight, where aesthetic enhancements like fog effects heighten the dramatic tension. Finally, players can achieve a 'Perfect Graduate' ending by successfully saving more than 80% of the students, adding replayability and rewarding skillful performance.

2.3 Graphics and Character Designing

To maintain visual cohesion across a frame-by-frame workflow, we utilized a multi-stage, AI-assisted pipeline. Gemini (Nano Banana 2) was leveraged for preliminary character drafts and conceptual sketches. For the narrative and environmental assets, ChatGPT Images 2.0 was employed to generate consistent, high-resolution backgrounds and the story-driven illustrations for the opening and ending scenes. These drafts were then refined into character behavior assets using a custom project built in Grok to ensure strict consistency in visual elements such as hairstyle, clothing, and skin tones. For the animation phase, Ludo AI was integrated to produce fluid behavioral frames by interpolating between keyframe inputs. To prepare these assets for the game engine, we used the iLoveIMG background remover to isolate the characters, which were then integrated into the Tuanjie animation timeline to ensure smooth transitions between complex behaviors. This hybrid approach successfully preserved the "rubber-hose" charm while significantly increasing production efficiency.

2.4 Coding

The project was developed within the Tuanjie Editor 1.8.1, which managed the game's core mechanics and 2D physics engine. We utilized Visual Studio Code integrated with GitHub Copilot to analyze scripts and generate optimized C# code for the player controller, camera, and UI systems. For version control and collaboration, we established a central repository on GitHub, which allowed each member to contribute independently while maintaining a clear modification history. Our development process followed an

Agile methodology, utilizing regular sprints to align our technical goals and coordinate the implementation of complex features like the Level 4 Infinite Scroller.

3. Game Elements

3.1 Character Designs (Abstract of sprites)

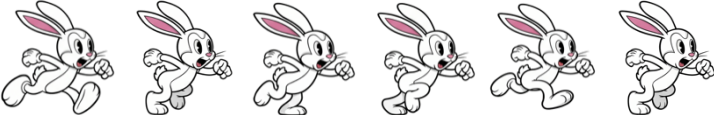
Main Character: Joe

State	Graphics
Idle	
Run	
Attack	
Jump and Fall	
Dash	
Get Hit	
Die	

Tony

Brew start and loop	
---------------------	--

Enemies

Character (State)	Graphics
Mover (Run)	
Mover (Get Hit and Die)	
Chaser (Run)	
Chaser (Get Hit and Die)	
Flyer (Fly)	
Flyer (Get Hit and Die)	

Boss

Boss (Phase 1)	
-------------------	--

Boss (Phase 2)	
Boss (Get hit and die)	

Table

Table (Idle, Block and Break)	
----------------------------------	--

Bubble & student

Student & Bubble (Idle and Bubble Pop)	
Student saved	

3.2 Environments / Levels / Screens

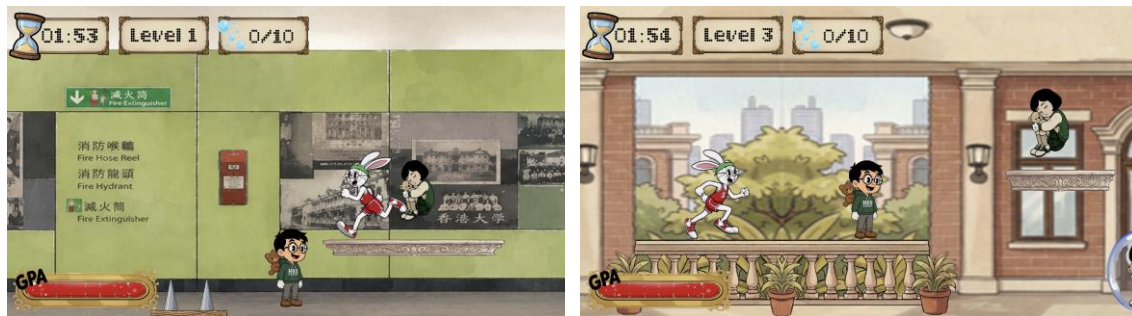
3.2.1 Opening, Ending, Start Menu, Buttons and Panels





After playing the opening trailer of the game, players get to the entry point of the game featuring the title *Rapocalypse – GPA Saver* in a stylized, wobbly rubber-hose font. Players can access the "Start Game" button, checkup settings, look for credits, or exit. This screen establishes the nostalgic aesthetic and introduces the energetic jazz soundtrack that persists throughout Joe's dream.

3.2.2 Round 1 & 3: Side-Scrolling Campus (Rescue)



Depicting the sprawling university campus, these rounds require Joe to navigate 2D platforms while fending off invading rabbits with sedative shots. The core objective is to utilize the Distance-Based Proximity mechanic to pop student bubbles and "free" them before they are touched by enemies. To increase difficulty, we implemented specialized "tricky" bubbles: some are positioned in high-reach areas requiring the "double dash jump" movement tech, while others are strategically hidden after teleport gates. These specialized placements serve as the trickiest part of the rescue missions; consequently, players are encouraged to attempt these rounds multiple times to master the traversal techniques and obtain the maximum number of bubbles for the best possible ending.

3.2.3 Round 2: Fixed Defense



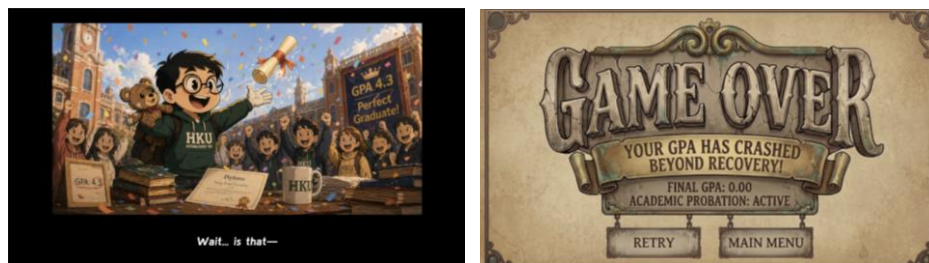
This scene takes place in a static environment where Joe must protect Tony, who is brewing the antidote in the center of the screen. Players must manage enemy waves spawning from both the left and right sides of the laboratory. Success is determined by Tony finishing the potion before either his or Joe's GPA bar hits zero.

3.2.4 Round 4: Boss Arena



The final environment is a dramatic, high-stakes arena where Joe faces the Rabbit Queen. The background shifts into a chaotic, fragmented representation of an exam hall filled with flying papers and ink. Players must utilize the dash and double-jump mechanics to survive the Queen's multi-phase attack patterns, with the difficulty scaling based on the number of students rescued in previous rounds. This final showdown is visually enhanced by an Infinite Scroller background and fog effects that emphasize the disjointed, high-pressure atmosphere of Joe's academic nightmare.

3.2.5 Game Over / Ending Screen



If Joe's GPA reaches 0.0 or the time runs out, the GameOverUI is triggered via the UIManager, freezing time and allowing the player to "Reset" (restarting the current level). Player can choose to restart the current levels or return to the main menu. Upon victory, the corresponding ending is played based on the total number of SavedStudents pulled from the GameManager. If the player saves over 80% of the students, they are presented with the "Perfect Graduate" ending; If the player saves under 25% of the student, they will be showed the bad ending; otherwise, they receive a standard ending.

3.3 UI, Controls, and Feedback

Move: WASD / Arrow Keys, Jump: Space (double jump), Dash: Shift, Shoot: Left Mouse, Pause: Esc

3.4 Audio

The audio design is central to establishing the game's nostalgic 1930s "rubber-hose" aesthetic, so selected authentic ragtime and vintage jazz tracks. All background music and sound effects (SFX) were sourced externally using a paid premium subscription to the audio library, [Uppbeat](#).

4. Evaluation: The Four Elements (Jesse Schell)

4.1 Mechanics

The mechanics of our game translate academic stress into high-stakes action. The gameplay includes four rounds, following a side-scroller/defense hybrid model.

Health as Currency (GPA System): Traditional health points are replaced by a GPA bar (starting at 4.3). Damage from "Rabbit" enemies results in a GPA deduction, the amount of which depends on the enemy type. This reinforces the link between survival and academic performance. **The "Vision" Saving Mechanic:** To simulate the fragility of focus, Joe frees trapped students from protective bubbles. Unlike traditional collectibles, these bubbles use a distance-based proximity trigger; they pop automatically when Joe approaches, requiring the player to actively defend the newly vulnerable students from nearby rabbits. This vision check also applies to "Chaser" type enemies, which will pursue the player indefinitely once they "see" the player, and serves as the boss fight trigger. In the final round, the player passes through a fog and encounters the boss; the boss fight logic is only triggered when the boss "sees" the player. Bubble that trapped the student will pop out once the player get close. **Escort and Defense:** Unlike the traditional side-scrolling gameplay of Rounds 1 and 3, Round 2 shifts mechanics to a fixed defense. Here, the player must protect an NPC (Tony) while managing enemy waves from two directions, testing spatial awareness and crowd control. **The "Perfect Graduate" Goal:** A secondary mechanic tracks the total SavedStudents via a persistent Singleton GameManager, determining the final narrative outcome.

4.2 Aesthetics

The game utilizes visual nostalgia to contrast with the modern anxiety of university life:

Art Direction: The game adopts a 1930s "rubber-hose" animation style (inspired by Cuphead). This aesthetic includes exaggerated character physics, noodle-like limbs, and expressive "pie-cut" eyes for both Joe and the "Rabbit" enemies.

Visual UI Feedback: The HUD is designed to mirror a student's academic dashboard. The Health Bar (GPA) changes color dynamically (Green → Yellow → Red) via the UIManager as the GPA drops, providing immediate emotional feedback to the player.

Animation and Audio: Sprite-based animations are used for smooth "pop" effects on bubbles and specific "stuck," "saved," or "death" states for students. The audio landscape features fast-paced, jazz-inspired tracks that intensify during the Rabbit Queen boss fight to heighten dramatic tension.

Inspiration: The rabbit was chosen as the iconic enemy for its symbolic roots in Alice in Wonderland. This serves as a metaphor for the game's narrative: Joe is essentially "down the rabbit hole," suggesting that his experience may just be a dream.

4.3 Story

The narrative frames the entire game as a lucid dream triggered by Joe's total exhaustion. **The Premise:** Joe is a student who has been pushing himself to the limit to improve his low GPA. After overworking himself while studying for his last final, he collapses into a deep sleep right before the big day. In his dream, he finds himself in an apocalypse of "Rabbits" that represent the way stress multiplies when left unchecked. Ironically, these rabbits threaten to cancel the very exams Joe needs to pass to succeed. **The Conflict:** To wake up and reach his goal, Joe must face the "Rabbit Attack." He is tasked with defeating the rabbit waves and rescuing trapped students to ensure the final exam can actually take place. This journey culminates in a showdown with the Rabbit Queen, the physical personification of final exam pressure. **Narrative Delivery:** Rather than using heavy dialogue, the story is told through environmental cues and fragmented transitions that simulate the hazy, disjointed feeling of a dream.

4.4 Technology

Developed in Unity, the game leverages 2D-specific systems to ensure a responsive feel.

State Management: The game uses a Singleton GameManager that persists across scenes (DontDestroyOnLoad), managing the global student count and handling asynchronous scene transitions with a dedicated transitionAnimator.

Physics and Collision: High-speed projectiles (sedative shots) and movement are handled by Rigidbody2D and BoxCollider2D. Traps and bubbles utilize Trigger events to allow fluid movement without halting player momentum. We also included some Playerwalls that only block the way of players but not enemy, increasing the difficulty of the game.

AI Architecture: Mover (Highest health, slow movement, low damage to player; horizontal movement only, rebound on walls and obstacles, could initialize its direction base on player position), Chaser (Moderate health, fast movement, moderate damage to player; Detect and chase the player once he enters its “vision”, can detect player sprite and change its direction, can jump over obstacles) and Flyer (Move fastest yet with minimal health, cause highest damage, can pass through obstacles, do not be affected by gravity, can detect player sprite and change its direction).

Boss (Rabbit Queen): Implemented via Animator State Transitions and Trigger-based Logic, allowing her to cycle through phases (spawning minions at different rates and difficulties, animation performances) based on health thresholds.

InfiniteScroller: Function for Round 4 to allow automagical generation of background and obstacles.

Optimization: To maintain performance on Web and Windows targets, bubbles utilize an Update-based distance check and OnBecameVisible/Invisible logic to toggle animations only when relevant to the player's "vision." Same logic as Chaser vision and BossFightTrigger, which is based on the player position to perform corresponding actions.

5. Gameplay Testing and Issue Analysis

5.1 Testing Methodology

To ensure the stability of our core mechanics, our team conducted rigorous gameplay testing, primarily focusing on stress-testing the physics engine and input systems. Our methodology included:

- **Boundary testing:** attempting to jump out of the playable map area
- **Continuous movement tests:** holding/spamming dash in a single direction
- **Input-spamming:** pressing varied combinations of keys rapidly to force sequence-breaking bugs

Through this process, we identified several unexpected behaviors.

5.2 Issue Analysis & Resolutions

- **Issue 1: "Dash-Jump" Momentum Anomaly:**
 - During input-spam testing, we discovered that executing a "Dash" immediately followed by a "Jump" command caused the character's vertical and horizontal velocity to stack, resulting in an abnormally high jump.

- Instead of patching this out, we recognized this as a highly engaging "movement tech." Similar to movement exploits found in competitive games like Apex Legends (tap strafing), this mechanic raises the skill ceiling.
- We officially transitioned this bug into a feature and designed specific areas that can only be accessed by players who master this hidden technique.
- **Issue 2: Animation Latency and Directional Desynchronization**
 - During combat testing, we initially identified a "double-attack" bug caused by overlapping a code-based Coroutine timer with an Animation Event trigger.
 - The problem stems from the 0.5-second wind-up in the shooting animation: the projectile's trajectory is calculated at the end of the wind-up, meaning if the player changes direction (e.g., turning left) during the delay after inputting an attack (e.g., facing right), the projectile fires erroneously to the new direction. This delay also makes the game feel sluggish.
 - The planned fix is to capture and store the player's facing direction at the exact moment the input is registered, rather than waiting for the animation to conclude.

6. Future Plan

6.1 Technical Upgrades and Refinements

- **AI Animation Refinement:** While leveraging AI tools to generate character animations accelerated our initial prototyping, it presented distinct limitations that require future manual refinement. Specifically, the AI struggled with spatial awareness and inverse kinematics. To resolve this, our future roadmap includes manually editing the keyframes of these AI-generated animations using Unity's built-in Animation window, or utilizing Unity's IK rigging system to force the hands into the correct alignment with the weapon.
- **Core Systems:** We will implement a robust save system and perform general optimization and code refactor to enhance performance and maintainability

6.2 Advanced Level Design & Environmental Mechanics

- **Level Design & Exploration:** We will overhaul the level design to be more deliberate and interactive, including:

- **Intentional Obstacle Placement:** Designing platforms and enemy encounters that require the player to think critically about spacing and timing.
- **Balance:** Extensive balance tweaks will be conducted for combat, enemy difficulty, and item effectiveness based on user testing.
- **Exploration Elements:** Introducing more levels or hidden levels such as hidden teleports, and secret areas to reward players who explore the map boundaries and master the game's techniques.
- **User Interface and Onboarding:** We will implement UI improvements and create better tutorials to streamline player onboarding.
- **Expanded Features:** We plan to introduce **new modes** of play and implement **new items / abilities** that offer players varied strategic options.

7. Conclusion

Rapocalypse – GPA Saver successfully realizes its vision as a thematically rich, 2D action-platformer that translates the psychological pressure of university life into engaging, high-stakes gameplay. Heavily inspired by the distinctive 1930s rubber-hose aesthetic of Cuphead, the project's greatest strength lies in its core mechanical innovation: directly linking player survival to academic performance through the GPA health bar.

Throughout the development cycle, our team successfully implemented a diverse range of gameplay experiences across four distinct levels that effectively mirrors the escalating stress of an exam period. Utilizing Unity's 2D physics engine alongside an AI-assisted asset pipeline allowed us to efficiently bring our "academic apocalypse" to life, creating unique state-driven enemy behaviors.

While the rapid prototyping process and rigorous playtesting revealed certain technical limitations—such as the nuances of refining AI-generated animations and boundary handling in the infinite scroller—these challenges provided valuable insights into game optimization. In fact, anomalies like the dash-jump momentum were successfully adapted into high-skill gameplay features. Ultimately, the project establishes a strong proof-of-concept that merges aesthetic nostalgia with relatable, modern thematic content, laying a solid foundation for future technical refinements and expanded level designs.

References / Credits

The Cuphead game page (<https://cupheadgame.com/>)

Gemini (nano banana 2)

Chatgpt (image 2.0)

Tuanjie 1.8.1

Uppbeat (<https://uppbeat.io/music>)

Ludo ai (<https://ludo.ai/>)

Github

iLoveIMG background remover (<https://www.iloveimg.com/remove-background>)